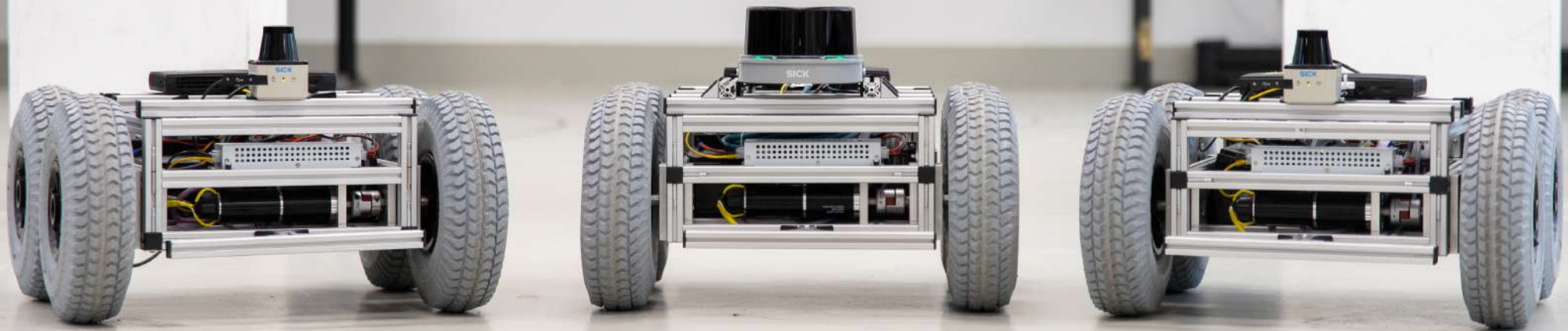


Robotik

Prof. Dr. Thomas Wiemann - FB AI - Robotik

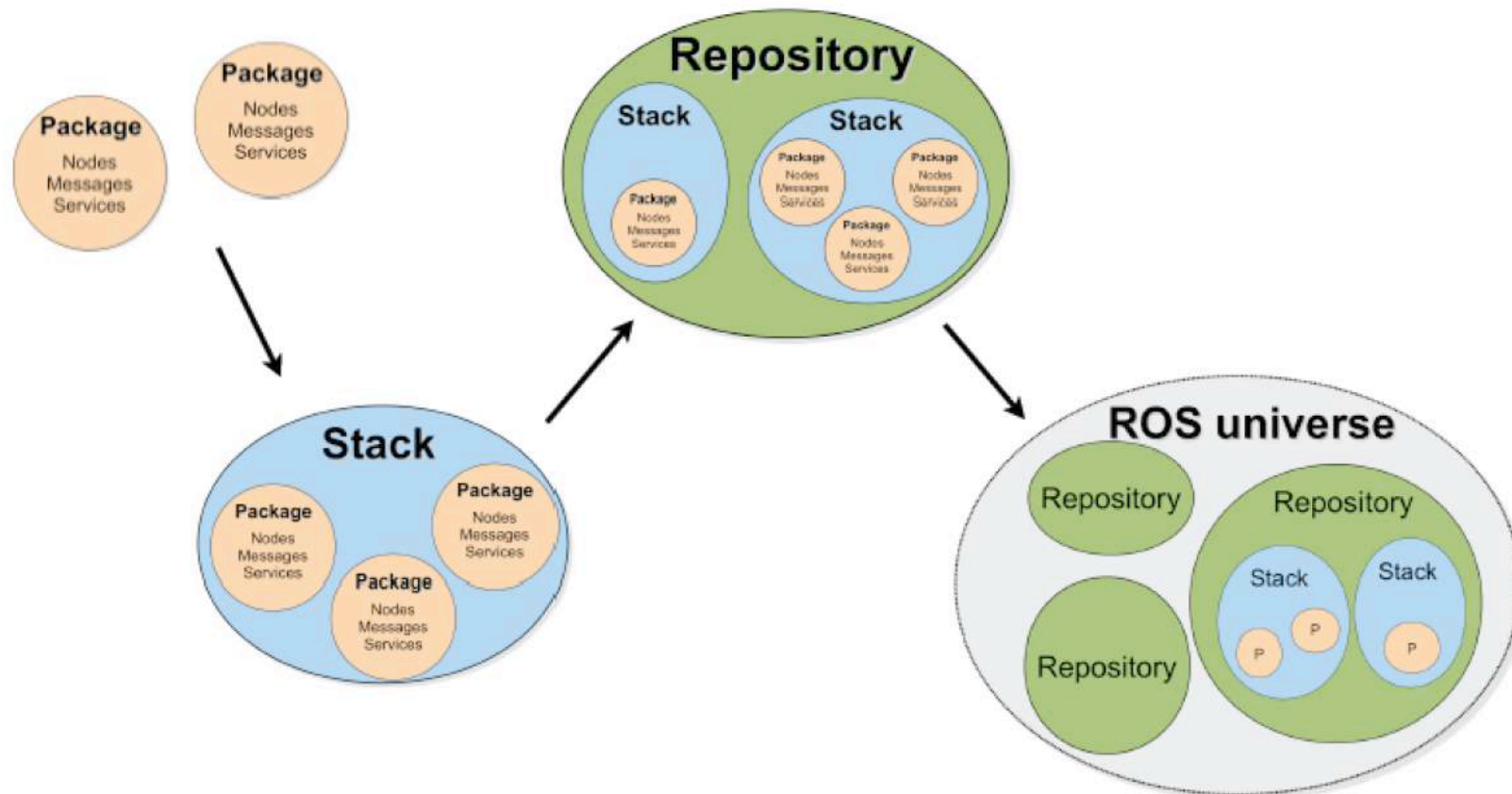


Hochschule Fulda
University of Applied Sciences



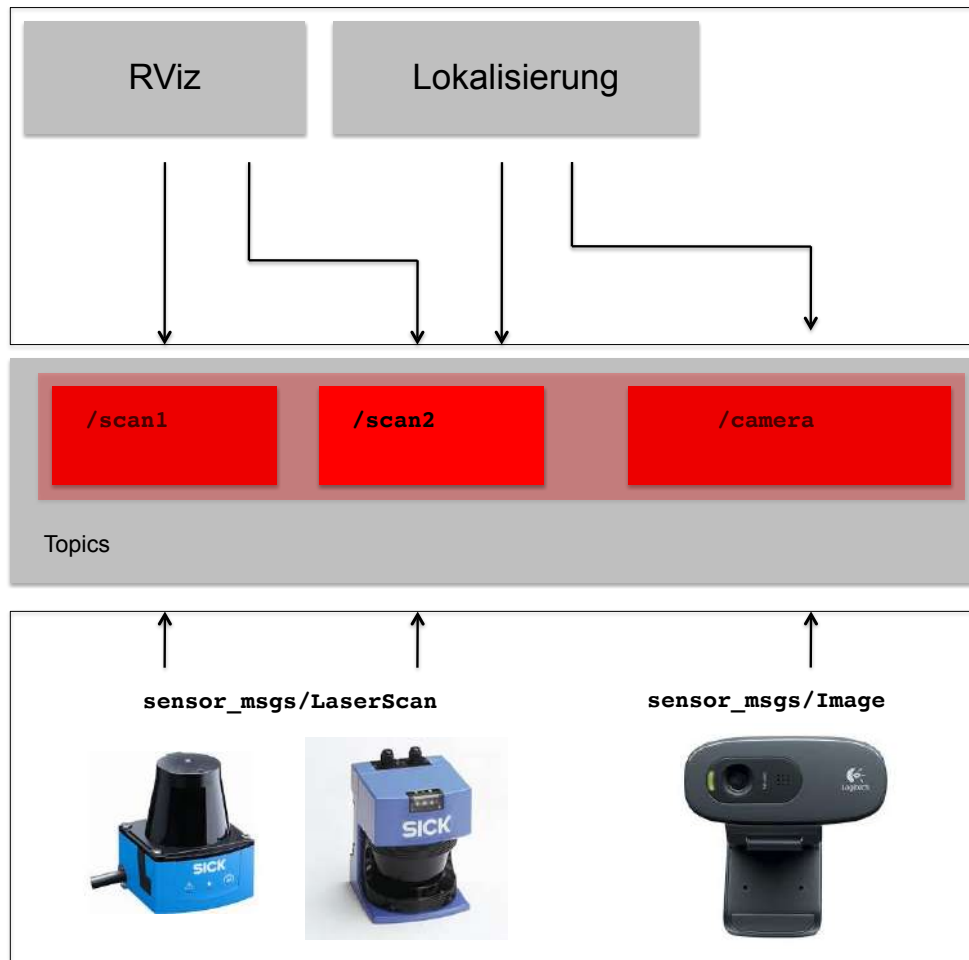


Das ROS - Ökosystem





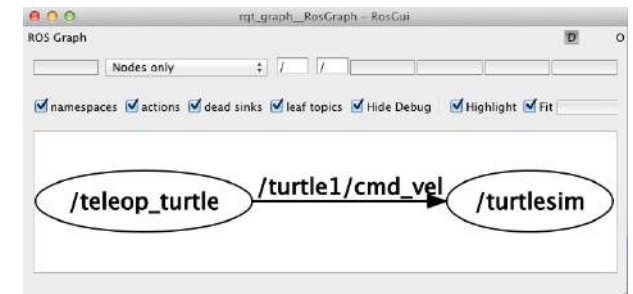
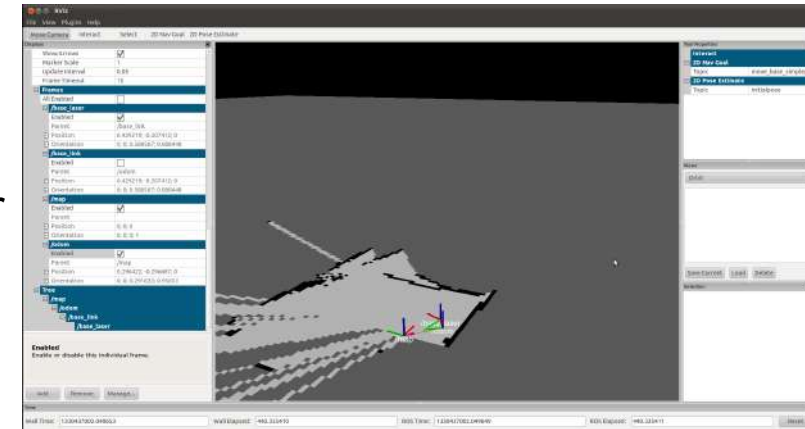
ROS Messaging (1)



Subscriber

DDS

Publisher





ROS Messaging (2)

```
# Single scan from a planar laser range-finder
#
# If you have another ranging device with different behavior (e.g. a sonar
# array), please find or create a different message, since applications
# will make fairly laser-specific assumptions about this data

Header header          # timestamp in the header is the acquisition time of
                        # the first ray in the scan.
                        #
                        # in frame frame_id, angles are measured around
                        # the positive Z axis (counterclockwise, if Z is up)
                        # with zero angle being forward along the x axis

float32 angle_min      # start angle of the scan [rad]
float32 angle_max      # end angle of the scan [rad]
float32 angle_increment # angular distance between measurements [rad]

float32 time_increment  # time between measurements [seconds] - if your scanner
                        # is moving, this will be used in interpolating position
                        # of 3d points
float32 scan_time       # time between scans [seconds]

float32 range_min       # minimum range value [m]
float32 range_max       # maximum range value [m]

float32[] ranges        # range data [m] (Note: values < range_min or > range_max should be discarded)
float32[] intensities   # intensity data [device-specific units]. If your
                        # device does not provide intensities, please leave
                        # the array empty.
```



ROS Messaging (3)

- ▶ Publisher geben kein Feedback, nachrichten können “verloren gehen”
- ▶ Andere ROS-Konstrukte zur Kommunikation zwischen Nodes:

Type	Strengths	Weaknesses
Message	• Good for most sensors (streaming data)	• Messages can be <u>dropped</u> without knowledge • Easy to overload system with too many messages
Service	• Knowledge of missed call • Well-defined feedback	• Blocks until completion • Connection typically re-established for each service call (slows activity)
Action	• Monitor long-running processes • Handshaking (knowledge of missed connection)	• Complicated • Mixed feedback from multiple action servers (not for SimpleAction)

http://events.ccc.de/congress/2014/Fahrplan/system/attachments/2497/original/How_I_Learned_to_Stop_Reinventing_and_Love_the_Wheels-bihlmaier-expanded.pdf



- ▶ In ROS2 lassen sich mehrere Nodes gleichzeitig starten
- ▶ Dazu gibt es sog. Launch-Files
- ▶ Programmierung über Python
- ▶ Vielfältige Konfigurationsmöglichkeiten, wir beschränken und hier auf das Wesentliche
- ▶ Alles Nodes in einem Launch-File werden nacheinander gestartet

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    ld = LaunchDescription()
    talker_node = Node(
        package="demo_nodes_cpp",
        executable="talker",
    )
    listener_node = Node(
        package="demo_nodes_py",
        executable="listener"
    )

    ld.add_action(talker_node)
    ld.add_action(listener_node)
    return ld
```



ROS - Launch Files

- ▶ Starten mit `ros2 launch <package> <launch_file>`
- ▶ Nodes können Start-Parameter haben
- ▶ Diese können in den Launch-Files definiert werden:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='python_parameters',
            executable='minimal_param_node',
            name='custom_minimal_param_node',
            output='screen',
            emulate_tty=True,
            parameters=[
                {'my_parameter': 'earth'}
            ]
        )
    ])
```



Topic-Remapping



- ▶ Problem: Zwei identische Sensoren/ Aufgaben auf dem Roboter.
- ▶ Wir wollen dafür die gleichen Nodes nehmen und nichts neu programmieren.
- ▶ Lösung: Topic Remapping
- ▶ Mit remapping kann man im Launch-File das Topic ersetzen, unter dem die Node normalerweise publishen würde.



```
sick_1 = Node(  
    package="sick_lms",  
    executable="sick_driver",  
    name="sick_1",  
    remappings=[  
        ("laser", "laser_1")  
    ]  
)
```



Remapping von /laser nach /laser_1

```
sick_2 = Node(  
    package="sick_lms",  
    executable="sick_driver",  
    name="sick_2",  
    remappings=[  
        ("laser", "laser_2")  
    ]  
)
```



Remapping von /laser nach /laser_2



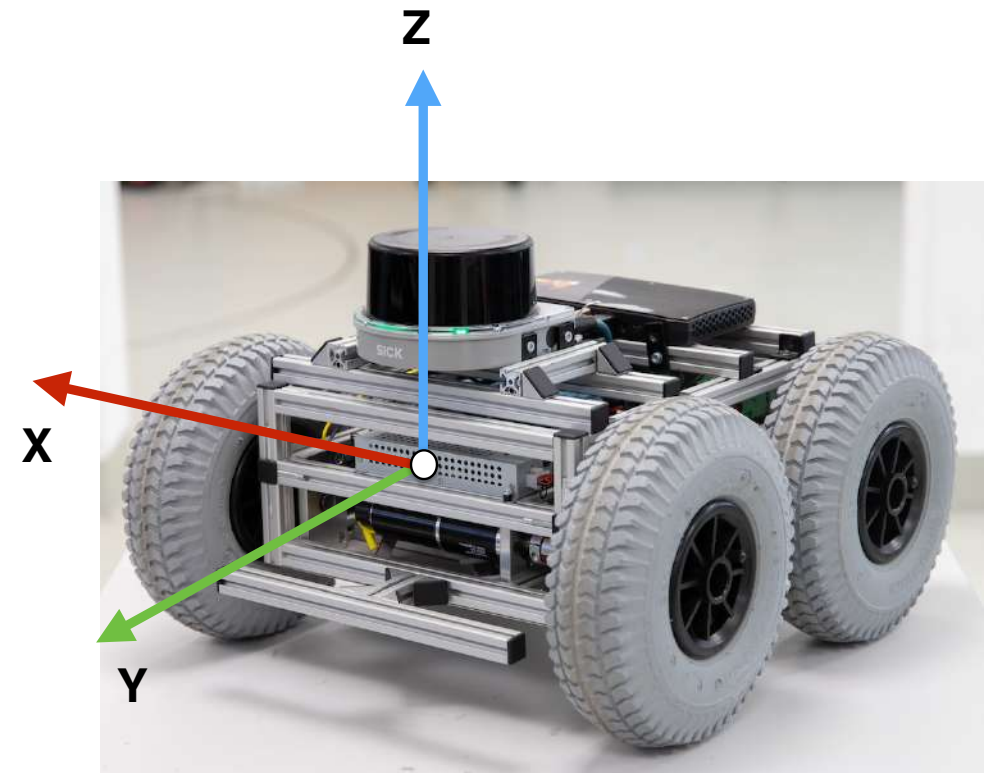
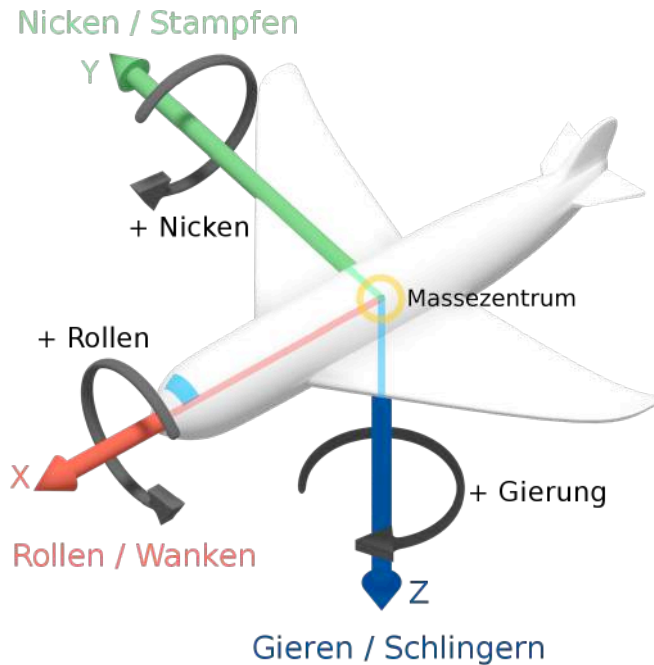
Gliederung

2. Robot Operating System

1. Grundlagen
2. Nachrichten
- 3. Transformationen**
4. Robotermodellierung
5. Mapping
6. Navigation
7. Ausblick



► Koordinatensysteme:

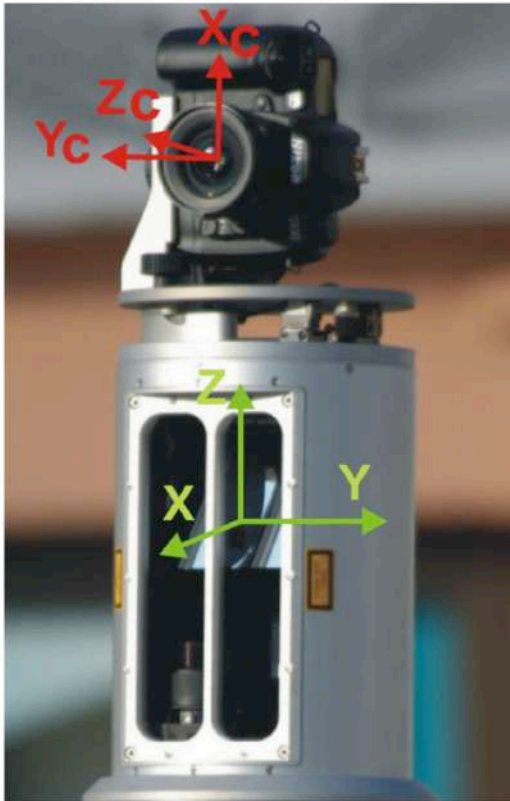


ROS' Koordinatensystem ist rechtshändig

Es gibt auch andere Konventionen. Im Buch von Hertzberg / Nüchter / Lingemann wird durchgängig ein linkshändiges Koordinatensystem verwendet. Entsprechend sind alle Formeln in linkshändischen Koordinaten. Umrechnung erforderlich!

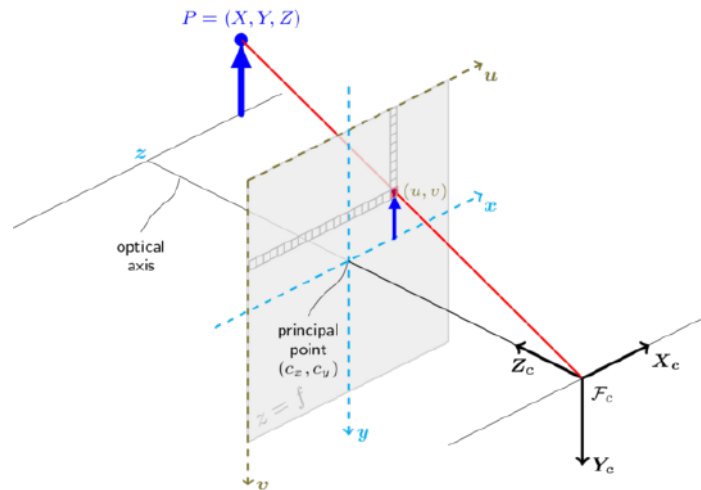


Koordinatensysteme - Beispiele



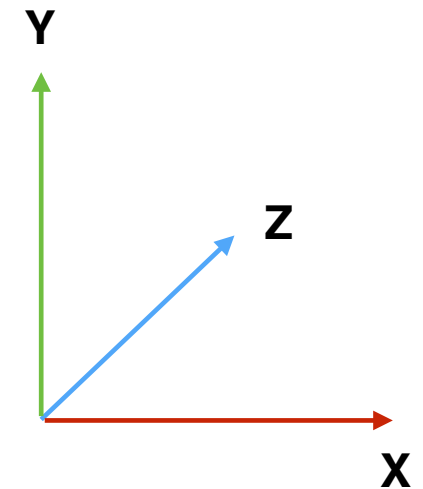
Riegl LMS

(right-handed)



OpenCV

(right-handed)

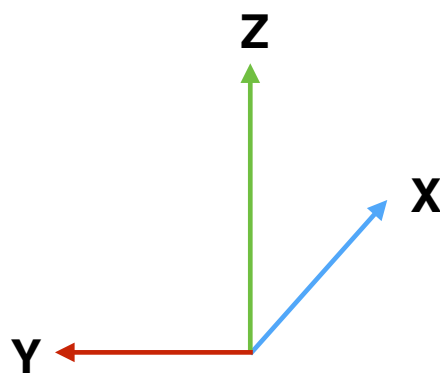


3DTK / Buch

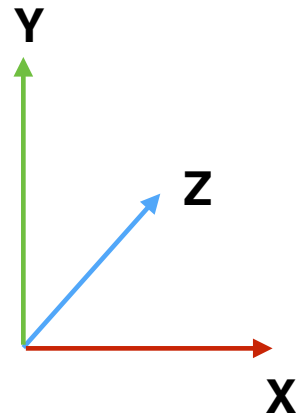
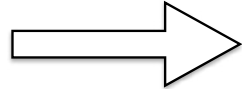
(left-handed)



► Beispiel Riegl → 3DTK



Riegl (in mm)



3DTK (in cm)

$$\begin{pmatrix} x_{3DTK} \\ y_{3DTK} \\ z_{3DTK} \end{pmatrix} = \begin{pmatrix} -0.1 \cdot y_{Riegl} \\ 0.1 \cdot z_{Riegl} \\ 0.1 \cdot x_{Riegl} \end{pmatrix}$$



Rotationen (1)

- ▶ Rotationen lassen sich mit Hilfe von Matrix / Vektormultiplikation realisieren
- ▶ Für 2D-Rotationen gilt:

$$R_{\alpha} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}$$

- ▶ Rotationsmatrizen in 3D:

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}$$

$$R_y(\alpha) = \begin{pmatrix} \cos \alpha & 0 & \sin \alpha \\ 0 & 1 & 0 \\ -\sin \alpha & 0 & \cos \alpha \end{pmatrix}$$

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Herleitung? Koordinatensystem??



Rotationen (2)

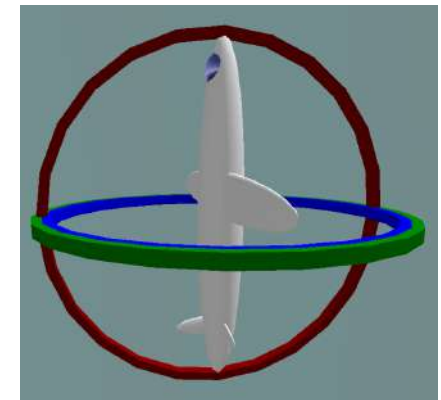
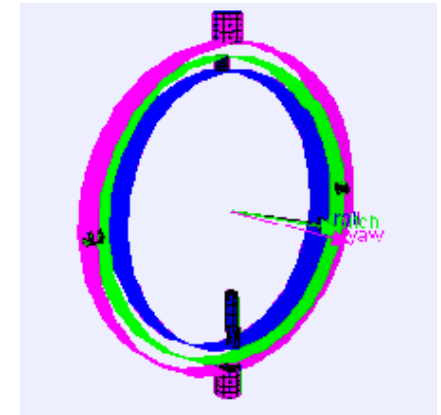
- ▶ Rotationen um drei Achsen lassen sich durch Verkettung der Drehmatrizen realisieren

$$\mathbf{R}_{\alpha,\beta,\gamma} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- ▶ Bei $\beta = \pi/2$ tritt der sogenannte “Gimbal Lock” auf

- ▶ Dann gilt $\cos \beta = 0$ und $\sin \beta = 1$

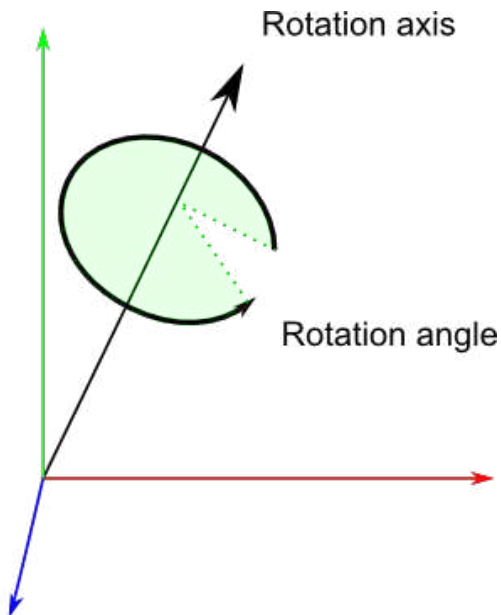
$$\begin{aligned} \mathbf{R} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 \\ \sin \alpha \cos \gamma + \cos \alpha \sin \gamma & -\sin \alpha \sin \gamma + \cos \alpha \cos \gamma & 0 \\ -\cos \alpha \cos \gamma + \sin \alpha \sin \gamma & \cos \alpha \sin \gamma + \sin \alpha \cos \gamma & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 \\ \sin(\alpha + \gamma) & \cos(\alpha + \gamma) & 0 \\ -\cos(\alpha + \gamma) & \sin(\alpha + \gamma) & 0 \end{bmatrix} \end{aligned}$$





Rotationen (3)

- ▶ Rotationen lassen sich in der Darstellung als Quaternionen ohne Locks verketteten
- ▶ Zahl $\mathbf{q} = q_r + q_i\mathbf{i} + q_j\mathbf{j} + q_k\mathbf{k}$ mit $i^2 + j^2 + k^2 = ijk = -1$
- ▶ Einheitsquaternionen repräsentieren eine Rotation als “Axis-Angle” (θ, x, y, z)



$$q = \begin{pmatrix} \cos \theta/2 \\ x \sin \theta/2 \\ y \sin \theta/2 \\ z \sin \theta/2 \end{pmatrix}$$

$$(x, y, z) = \frac{q_i}{\sin \theta/2}, \frac{q_j}{\sin \theta/2}, \frac{q_k}{\sin \theta/2}$$
$$\theta = 2 \cos^{-1}(q_r)$$

Alle notwendigen Operationen sind in ROS und anderen Umgebungen implementiert. Hier nur als Hinweis zur Einordnung. Vollständiges gutes Tutorial gibt es hier:

<https://danceswithcode.net/engineeringnotes/quaternions/quaternions.html>



Homogene Koordinaten

- ▶ Rotation, Skalierung und Scherung eines 3D-Punktes lassen sich durch 3x3-Matrizen ausdrücken
- ▶ Problem: Translation erfordert eine Vektor-Addition
- ▶ Lösung: Erweiterung des Raumes auf 4-Dimensionen
- ▶ Nun lassen sich auch Translationen durch Matrixoperationen beschreiben:

$$(x, y, z)^T \mapsto (x, y, z, w)^T$$

- ▶ Rücktransformation von homogenen Koordinaten

$$(x', y', z', w')^T \mapsto \left(\frac{x'}{w'}, \frac{y'}{w'}, \frac{z'}{w'} \right)^T$$

- ▶ Beispiel Translation

$$Tx = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \end{pmatrix}$$



Transformationsmatrizen

► Wichtige Transformationen:

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translation

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um x-Achse

$$\begin{pmatrix} \cos \beta & 0 & \sin \beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \beta & 0 & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um y-Achse

$$\begin{pmatrix} \cos \gamma & -\sin \gamma & 0 & 0 \\ \sin \gamma & \cos \gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um z-Achse

► Achtung: Darstellungen hängen vom Koordinatensystem ab!!

► Weitere Transformationen:

$$\begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Skalierung

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 0 \end{pmatrix}$$

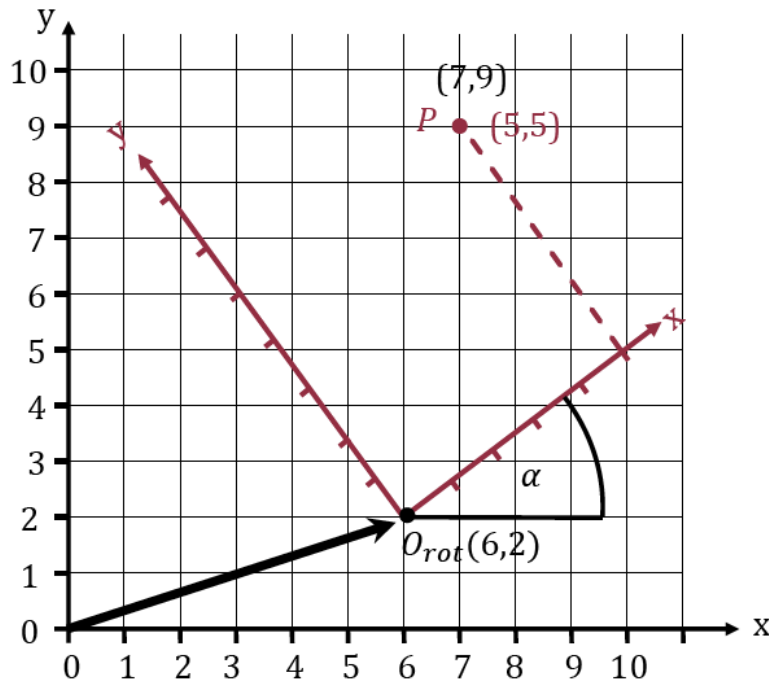
Perspektive

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Orthogonale Projektion



Basiswechsel (1)



- Beschreibe P im schwarzen Koordinatensystem
- Suche Matrix, die den P_{rot} nach P_{schw} transformiert
- Drehe schwarzes System um α :
 - $\cos(\alpha) = 4 / 5 = 0.8$
 - $\sin(\alpha) = 3 / 5 = 0.6$
- Rotations- und Translationsmatrix:

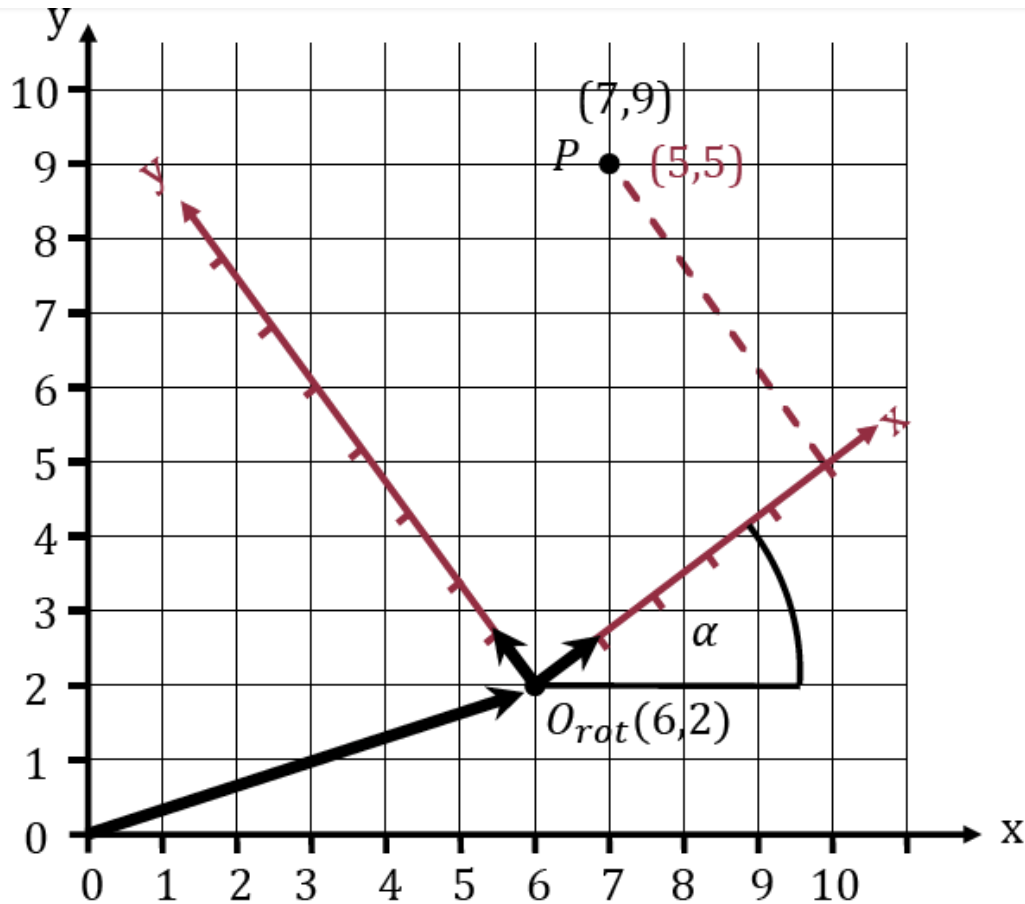
$$R(\alpha) = \begin{pmatrix} 0.8 & -0.6 & 0 \\ 0.6 & 0.8 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad T_{rot} = \begin{pmatrix} 1 & 0 & 6 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix}$$

- Komplette Transformation:

$$P_{schw} = T_{rot} \cdot R(\alpha) \cdot P_{rot} = \begin{pmatrix} 0.8 & -0.6 & 6 \\ 0.6 & 0.8 & 2 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 5 \\ 5 \\ 1 \end{pmatrix} = \begin{pmatrix} 7 \\ 9 \\ 1 \end{pmatrix}$$



Basiswechsel (2)



e_y von rot, beschrieben in schwarz

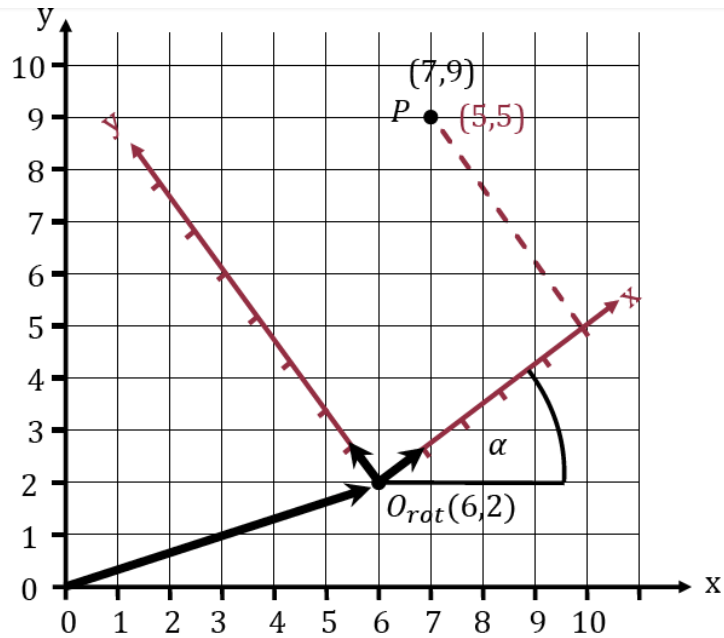
O von rot, beschrieben in schwarz

$$M_{rot \rightarrow schw} = \begin{pmatrix} 0.8 & -0.6 & 6 \\ 0.6 & 0.8 & 2 \\ 0 & 0 & 1 \end{pmatrix}$$

e_x von rot, beschrieben in schwarz



Basiswechsel (3)



► Test:

$$P_{rot} = M_{schw \rightarrow rot} \cdot P_{schw} = \begin{pmatrix} 0.8 & 0.6 & -6 \\ -0.6 & 0.8 & -2 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 7 \\ 9 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \\ 1 \end{pmatrix}$$

$$M_{schw \rightarrow rot} = M_{rot \rightarrow schw}^{-1}$$

$$M_{schw \rightarrow rot} = R(\alpha)^{-1} \cdot T(O_{rot})^{-1}$$

$$M_{schw \rightarrow rot} = R(-\alpha) \cdot T(-O_{rot})$$

$$M_{schw \rightarrow rot} = \begin{pmatrix} 0.8 & 0.6 & -6 \\ -0.6 & 0.8 & -2 \\ 0 & 0 & 1 \end{pmatrix}$$



Basiswechsel (4)

$$\begin{aligned}
 \mathbf{M}_{B \rightarrow A} \cdot \mathbf{p}_B = \mathbf{p}_A = & \begin{pmatrix} e_{(x,x)}^A & e_{(y,x)}^A & e_{(z,x)}^A & O_x^A \\ e_{(x,y)}^A & e_{(y,y)}^A & e_{(z,y)}^A & O_y^A \\ e_{(x,z)}^A & e_{(y,z)}^A & e_{(z,z)}^A & O_z^A \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} p_x^B \\ p_y^B \\ p_z^B \\ 1 \end{pmatrix} \\
 & \begin{array}{cccc} \text{Punkt} & & & \text{Punkt} \\ \text{beschrieben in A} & e_x \text{ von B} & e_y \text{ von B} & e_z \text{ von B} \\ & \text{Ursprung von B} & & \text{beschrieben in B} \end{array} \\
 & \underbrace{\hspace{15em}}_{\text{Beschrieben in A}}
 \end{aligned}$$

$$(\mathbf{M}_{B \rightarrow A})^{-1} \cdot \mathbf{p}_A = \mathbf{M}_{A \rightarrow B} \cdot \mathbf{p}_A = \mathbf{p}_B$$

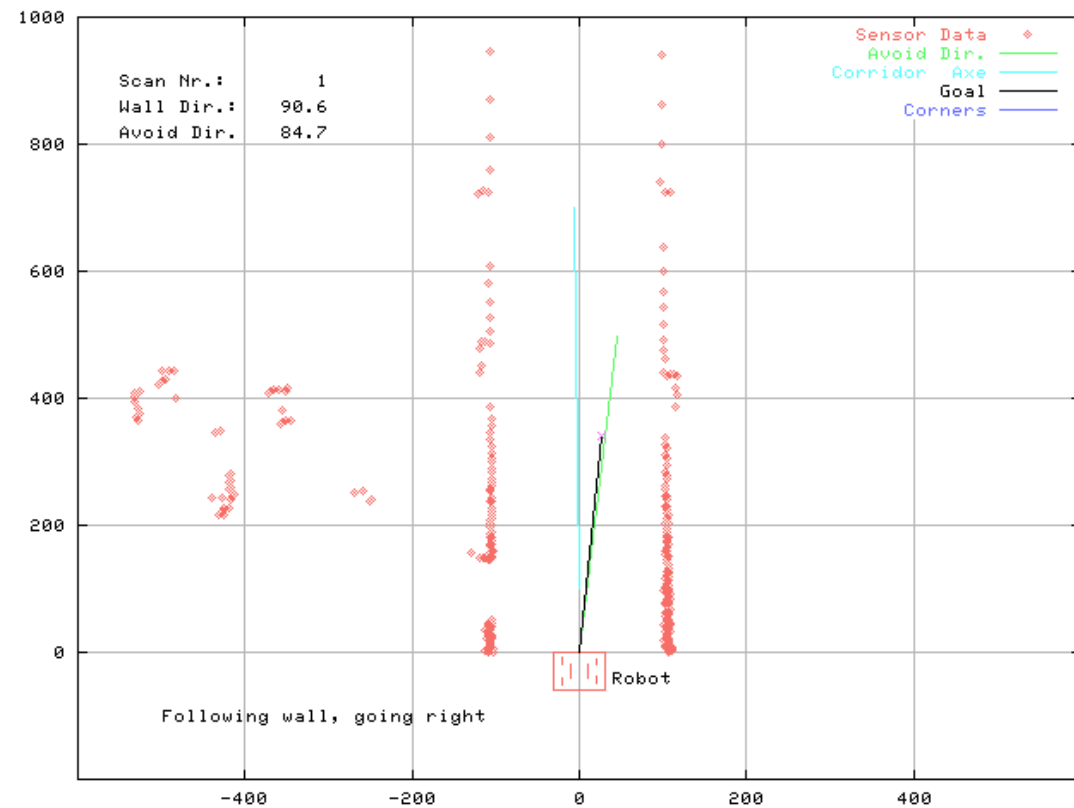


ROS - Transformationen (1)

- ▶ Jeder Sensor liefert seine Daten in einem eigenen lokalen Referenzsystem
- ▶ Typischerweise ist der Ursprung in der Mitte des Sensors



Beispiel SICK LMS200





ROS - Transformationen (2)

- ▶ In der Regel sind die Sensoren nicht in der Mitte des Roboters montiert
- ▶ Oftmals braucht man für verschiedene Aufgaben unterschiedliche Referenzsysteme



Zur Kollisionsvermeidung
sollten die Laser-Daten in das
Roboterkoordinatensystem
konvertiert werden!

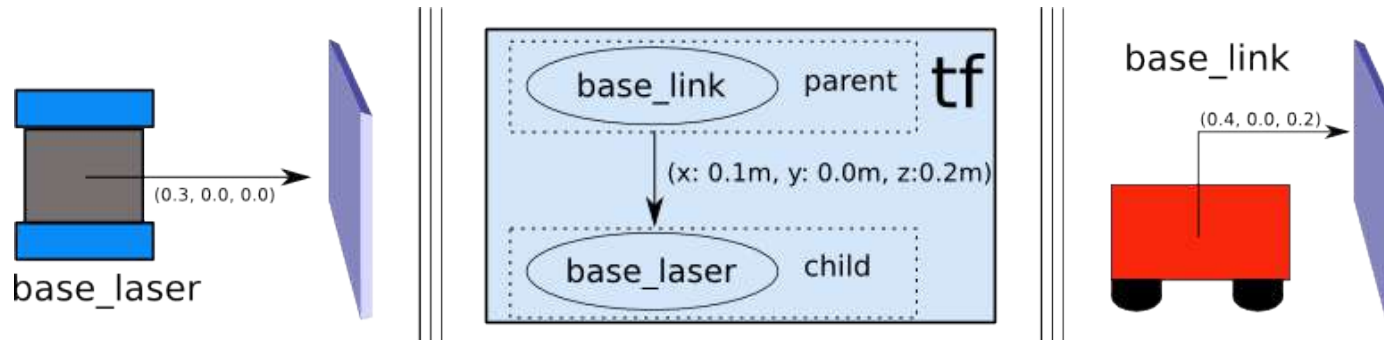
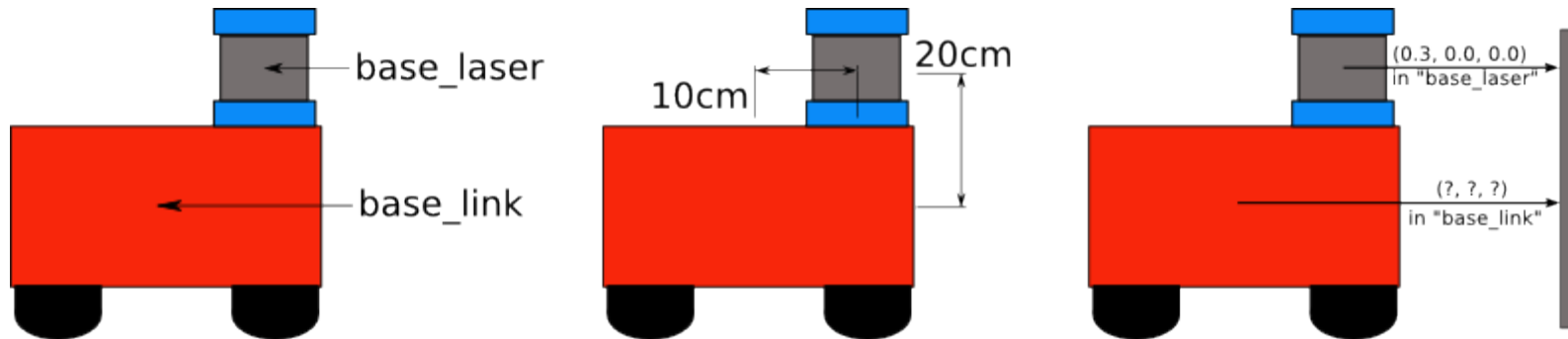


Wenn der Arm etwas greifen
soll, sollten die Messwerte
der 3D-Kamera relativ zu
seinem Ursprung sein



ROS Transformations (3)

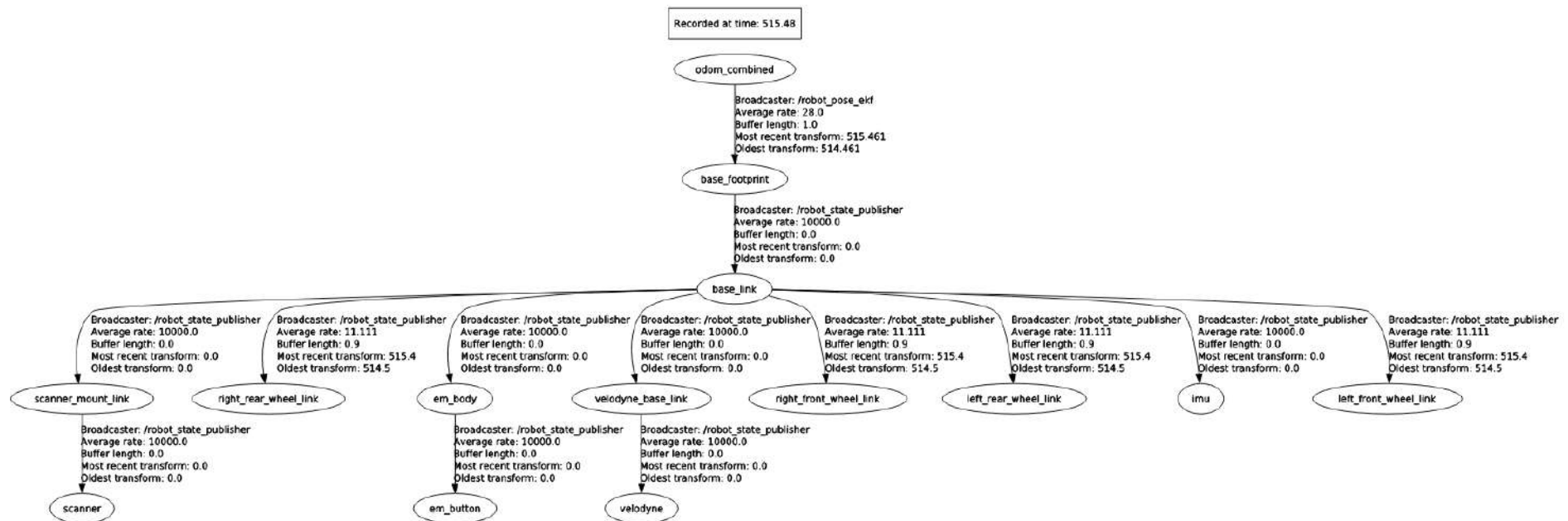
- ROS stellt via *tf* solche Transformationen zur Verfügung





ROS - Transformationen (4)

- ▶ Via `tf` lassen sich die Daten von einzelnen Komponenten automatisch Transformieren
- ▶ Es wird ein Transformations**baum** aufgespannt
- ▶ Es muss also immer ein Root-Koordinatensystem geben





ROS - Transformationen (5)





ROS - Transformationen (6)

